Тема 1: Установка MYSQL, создание БД и таблиц

Введение

В первые два десятилетия компьютерного века серверы среднего масштаба встречались достаточно редко. Компьютеры были дорогими изделиями, имевшимися в больших учреждениях, таких как банки и университеты. Громадные серверы, работавшие в режиме разделения времени, обеспечивали вычислительными мощностями целые корпорации. Большая часть использовавшегося на этих системах программного обеспечения была такой же грандиозной, как сами серверы. Поскольку единственный компьютер обслуживал нужды нескольких подразделений, если не всей организации, ему приходилось удовлетворять запросы всех пользователей.

На другом конце спектра располагался персональный компьютер. В результате революции, произведенной ПК, вместо одного компьютера в крупной компании появился один компьютер в каждом доме. Хотя мощность этих компьютеров легко позволяла удовлетворить потребности отдельного пользователя, между возможностями и стоимостью персональных вычислений и вычислений в масштабе организации существовал большой разрыв.

Более всего этот разрыв был очевиден в области управления данными. Приложения, работающие с базами данных на больших машинах, содержали в себе все мыслимые функции. Из-за многоцелевой сущности этого программного обеспечения в него включалась любая необычная функция, если она была нужна хотя бы одному пользователю. Появились также приложения баз данных, удовлетворяющие потребности управления данными для отдельных пользователей. Однако если базы данных для мэйнфреймов были слишком громоздки для потребностей среднего уровня, то возможности персональных баз данных были слишком ограниченны.

В первой половине 1990-х «жалкий» персональный компьютер достиг такого уровня, когда фактически оказался мощнее мэйнфреймов прежних лет. И если аппаратная часть более не была препятствием для вычислений среднего уровня, то таковым служило отсутствие доступного по цене программного обеспечения. Чтобы удовлетворить потребности некоммерческих организаций и малых предприятий в хранении данных, требовались доступные по цене операционная система и система управления базами данных. Одна половина этой задачи была решена с появлением таких дешевых и мощных операционных систем для серверов, как FreeBSD и Linux.

Решение той части задачи, которая относится к управлению базами данных, дают MySQL и mSQL. Они обладают мощью и гибкостью, сохраняя в то же время компактность и эффективность. В особенности MySQL удалось уместить большой набор возможностей в очень маленьком и быстром ядре. Хотя то и другое ядро далеко не обладают полным набором функций, имеющихся в дорогих корпоративных базах данных, они обладают вполне достаточными характеристиками, чтобы удовлетворить потребности управления базами данных среднего масштаба.

Основные идеи современной информационной технологии базируются на концепции, согласно которой данные должны быть организованы в базы данных с целью адекватного отображения изменяющегося реального мира и удовлетворения информационных потребностей пользователей. Эти базы данных создаются и функционируют под управлением специальных программных комплексов, называемых системами управления базами данных (СУБД). Увеличение объема и структурной сложности хранимых данных, расширение круга пользователей информационных систем привели к широкому распространению наиболее удобных и сравнительно простых для понимания реляционных (табличных) СУБД. Для обеспечения одновременного доступа к данным множества пользователей, нередко расположенных достаточно далеко друг от друга и от места хранения баз данных, созданы сетевые мультипользовательские версии СУБД. В них тем или иным путем решаются специфические проблемы параллельных

процессов, целостности (правильности) и безопасности данных, а также санкционирования доступа.

Ясно, что совместная работа пользователей в сетях с помощью унифицированных средств общения с базами данных возможна только при наличии стандартного языка манипулирования данными, обладающего средствами для реализации перечисленных выше возможностей. Таким языком стал SQL, разработанный в 1974 году фирмой IBM для экспериментальной реляционной СУБД System R. После появления на рынке двух пионерских СУБД этой фирмы - SQL/DS (1981 год) и DB2 (1983 год) - он приобрел статус стандарта де-факто для профессиональных реляционных СУБД. В 1987 году SQL стал международным стандартом языка баз данных, а в 1992 году вышла вторая версия этого стандарта.

Все языки манипулирования данными (ЯМД), созданные до появления реляционных баз данных и разработанные для многих систем управления базами данных (СУБД) персональных компьютеров, были ориентированы на операции с данными, представленными в виде логических записей файлов. Это требовало от пользователей детального знания организации хранения данных и достаточных усилий для указания не только того, какие данные нужны, но и того, где они размещены и как шаг за шагом получить их.

Рассматриваемый же ниже непроцедурный язык SQL (Structured Query Language - структуризованный язык запросов) ориентирован на операции с данными, представленными в виде логически взаимосвязанных совокупностей таблиц. Особенность предложений этого языка состоит в том, что они ориентированы в большей степени на конечный результат обработки данных, чем на процедуру этой обработки. SQL сам определяет, где находятся данные, какие индексы и даже наиболее эффективные последовательности операций следует использовать для их получения: не надо указывать эти детали в запросе к базе данных.

Для иллюстрации различий между ЯМД рассмотрим следующую ситуацию. Пусть, например, вы собираетесь посмотреть кинофильм и хотите воспользоваться для поездки в кинотеатр услугами такси. Одному шоферу такси достаточно сказать название фильма - и он сам найдет вам кинотеатр, в котором показывают нужный фильм. (Подобным же образом, самостоятельно, отыскивает запрошенные данные SQL.) Для другого шофера такси вам, возможно, потребуется самому узнать, где демонстрируется нужный фильм и назвать кинотеатр. Тогда водитель должен найти адрес этого кинотеатра. Может случиться и так, что вам придется самому узнать адрес кинотеатра и предложить водителю проехать к нему по таким-то и таким-то улицам. В самом худшем случае вам, может быть, даже придется по дороге давать указания: "Повернуть налево... проехать пять кварталов... повернуть направо...". (Аналогично больший или меньший уровень детализации запроса приходится создавать пользователю в разных СУБД, не имеющих языка SQL.)

Появление теории реляционных баз данных и предложенного Коддом языка запросов "alpha", основанного на реляционном исчислении, инициировало разработку ряда языков запросов, которые можно отнести к двум классам:

1. Алгебраические языки, позволяющие выражать запросы средствами специализированных операторов, применяемых к отношениям (JOIN - соединить, INTERSECT - пересечь, SUBTRACT - вычесть и т.д.).
2. Языки исчисления предикатов представляют собой набор правил для записи выражения, определяющего новое отношение из заданной совокупности существующих отношений. Другими словами исчисление предикатов есть метод определения того отношения, которое нам желательно получить (как ответ на запрос) из отношений, уже имеющихся в базе данных.

Разработка, в основном, шла в отделениях фирмы IBM (языки ISBL, SQL, QBE) и университетах США (PIQUE, QUEL. Последний создавался для СУБД INGRES (Interactive Graphics and Retrieval System), которая была разработана в начале 70-х годов в Университете шт. Калифорния и сегодня входит в пятерку лучших профессиональных СУБД. Сегодня из всех этих языков полностью сохранились и развиваются QBE (Query-By-Example - запрос по образцу) и SQL, а из остальных взяты в расширение внутренних языков СУБД только наиболее интересные конструкции.

В начале 80-х годов SQL "победил" другие языки запросов и стал фактическим стандартом таких языков для профессиональных реляционных СУБД. В 1987 году он стал международным стандартом языка баз данных и начал внедряться во все распространенные СУБД персональных компьютеров. Почему же это произошло?

Непрерывный рост быстродействия, а также снижение энергопотребления, размеров и стоимости компьютеров привели к резкому расширению возможных рынков их сбыта, круга пользователей, разнообразия типов и цен. Естественно, что расширился спрос на разнообразное программное обеспечение. Борясь за покупателя, фирмы, производящие программное обеспечение, стали выпускать на рынок все более и более интеллектуальные и, следовательно, объемные программные комплексы. Приобретая (желая приобрести) такие комплексы, многие организации и отдельные пользователи часто не могли разместить их на собственных ЭВМ, однако не хотели и отказываться от нового сервиса. Для обмена информацией и ее обобществления были созданы сети ЭВМ, где обобществляемые программы и данные стали размещать на специальных обслуживающих устройствах - файловых серверах.

СУБД, работающие с файловыми серверами, позволяют множеству пользователей разных ЭВМ (иногда расположенных достаточно далеко друг от друга) получать доступ к одним и тем же базам данных. При этом упрощается разработка различных автоматизированных систем управления организациями, учебных комплексов, информационных и других систем, где множество сотрудников (учащихся) должны использовать общие данные и обмениваться создаваемыми в процессе работы (обучения). Однако при такой идеологии вся обработка запросов из программ или с терминалов пользовательских ЭВМ выполняется на этих же ЭВМ. Поэтому для реализации даже простого запроса ЭВМ часто должна считывать из файлового сервера и (или) записывать на сервер целые файлы, что ведет к конфликтным ситуациям и перегрузке сети.

Для исключения указанных и некоторых других недостатков была предложена технология "Клиент-Сервер", по которой запросы пользовательских ЭВМ (Клиент) обрабатываются на специальных серверах баз данных (Сервер), а на ЭВМ возвращаются лишь результаты обработки запроса. При этом, естественно, нужен единый язык общения с Сервером и в качестве такого языка выбран SQL. Поэтому все современные версии профессиональных реляционных СУБД (DB2, Oracle, Ingres, Informix, Sybase, Progress, Rdb) и даже нереляционных СУБД (например, Adabas) используют технологию "Клиент-Сервер" и язык SQL. К тому же приходят разработчики СУБД персональных ЭВМ, многие из которых уже сегодня снабжены языком SQL.

Бытует мнение: Поскольку большая часть запросов формулируется на SQL, практически безразлично, что это за СУБД - был бы SQL.

Реализация в SQL концепции операций, ориентированных на табличное представление данных, позволило создать компактный язык с небольшим (менее 30) набором предложений. SQL может использоваться как интерактивный (для выполнения запросов) и как встроенный (для построения прикладных программ). В нем существуют:

- предложения определения данных (определение баз данных, а также определение и уничтожение таблиц и индексов);
- запросы на выбор данных (предложение SELECT);
- предложения модификации данных (добавление, удаление и изменение данных);

- предложения управления данными (предоставление и отмена привилегий на доступ к данным, управление транзакциями и другие). Кроме того, он предоставляет возможность выполнять в этих предложениях:
- арифметические вычисления (включая разнообразные функциональные преобразования), обработку текстовых строк и выполнение операций сравнения значений арифметических выражений и текстов;
- упорядочение строк и (или) столбцов при выводе содержимого таблиц на печать или экран дисплея;
- создание представлений (виртуальных таблиц), позволяющих пользователям иметь свой взгляд на данные без увеличения их объема в базе данных;
- запоминание выводимого по запросу содержимого таблицы, нескольких таблиц или представления в другой таблице (реляционная операция присваивания).
- агрегатирование данных: группирование данных и применение к этим группам таких операций, как среднее, сумма, максимум, минимум, число элементов и т.п.

В SQL используются следующие основные типы данных, форматы которых могут несколько различаться для разных СУБД:

INTEGER
- целое число (обычно до 10 значащих цифр и знак);

SMALLINT
- "короткое целое" (обычно до 5 значащих цифр и знак);

DECIMAL(p,q)
- десятичное число, имеющее p цифр ($0 < p < 16$) и знак; с помощью q задается число цифр справа от десятичной точки ($q < p$, если $q = 0$, оно может быть опущено);

FLOAT
- вещественное число с 15 значащими цифрами и целочисленным порядком, определяемым типом СУБД;

CHAR(n)
- символьная строка фиксированной длины из n символов ($0 < n < 256$);

VARCHAR(n)
- символьная строка переменной длины, не превышающей n символов ($n > 0$ и разное в разных СУБД, но не меньше 4096);

DATE
- дата в формате, определяемом специальной командой (по умолчанию mm/dd/yy); поля даты могут содержать только реальные даты, начинающиеся за несколько тысячелетий до н.э. и ограниченные пятым-десятым тысячелетием н.э.;

TIME
- время в формате, определяемом специальной командой, (по умолчанию hh.mm.ss);

DATETIME
- комбинация даты и времени;

MONEY
- деньги в формате, определяющем символ денежной единицы ($, руб, ...) и его расположение (суффикс или префикс), точность дробной части и условие для показа денежного значения.

В некоторых СУБД еще существует тип данных LOGICAL, DOUBLE и ряд других. СУБД INGRES предоставляет пользователю возможность самостоятельного определения новых типов данных, например, плоскостные или пространственные координаты, единицы различных метрик, пяти- или шестидневные недели (рабочая неделя, где сразу

после пятницы или субботы следует понедельник), дроби, графика, большие целые числа (что стало очень актуальным для российских банков) и т.п.

Ориентированный на работу с таблицами SQL не имеет достаточных средств для создания сложных прикладных программ. Поэтому в разных СУБД он либо используется вместе с языками программирования высокого уровня (например, такими как Си или Паскаль), либо включен в состав команд специально разработанного языка СУБД (язык систем dBASE, R:BASE и т.п.). Унификация полных языков современных профессиональных СУБД достигается за счет внедрения объектно-ориентированного языка четвертого поколения 4GL. Последний позволяет организовывать циклы, условные предложения, меню, экранные формы, сложные запросы к базам данных с интерфейсом, ориентированным как на алфавитно-цифровые терминалы, так и на оконный графический интерфейс (X-Windows, MS-Windows).

Установка и запуск MYSQL

Запустить установщик mySQL Server 5.0.

Запустить программу командами Пуск → MySQL → MySQL Server 5.0 → Command Line Client.

Создание новой БД

После набора пароля и появления в окне приглашения mysql>) набрать:

create database dbStud_Lessons;

Создание таблиц

Сначала необходимо сделать созданную БД, используемой по умолчанию:

mysql>use dbStud_Lessons;

Ответная реакция – сообщение: Database changed (рис. 1.1).

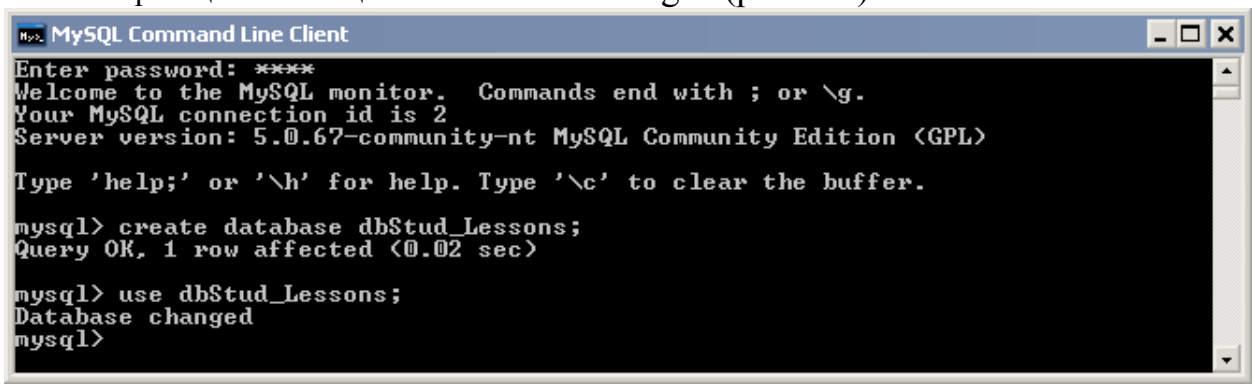

Рис. 1.1

Создать таблицу Student с ключевым полем KodStud:

создание таблицы Student с указанием типов данных полей:

```
create table student (
kodStud integer,
fam char (20),
imya char (15),
```

otch char (20),
gr_nomber integer);

Вид программного кода в командной строке представлен на рис. 1.2.

добавление ключевого поля:

ALTER [IGNORE] TABLE *table* ADD UNIQUE *[name] (column, ...)*

Идентично ADD INDEX за исключением того, что индексированные поля гарантированно уникальны. То есть, если пользователь попытается добавить значение, уже существующее в уникальном индексе, будет выдано сообщение об ошибке.

alter table student add unique kodStud (kodStud);

```
MySQL Command Line Client
mysql> create table student (
    -> kodStud integer,
    -> fam char(20),
    -> imya char(15),
    -> otch char(20),
    -> gr_nomber integer );
Query OK, 0 rows affected (0.13 sec)

mysql> alter table student add unique kodStud (kodStud);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql>
```

Рис. 1.2

*добавление поля **AUTO_INCREMENT**:*

Добавление нового числового столбца AUTO_INCREMENT (автоматическая нумерация) с именем c:

mysql> ALTER TABLE t2 ADD c INT UNSIGNED NOT NULL
AUTO_INCREMENT, ADD INDEX (c);

создание ключевого поля при создании таблицы:

Вид программного кода в командной строке представлен на рис. 1.3.

```
MySQL Command Line Client
mysql> create table example1 (
    -> kodStud integer,
    -> primary key (kodStud),
    -> fam char(20),
    -> imya char(15),
    -> otch char(20),
    -> gr_nomber integer);
Query OK, 0 rows affected (0.03 sec)

mysql> desc example1;
+-----------+----------+------+-----+---------+-------+
| Field     | Type     | Null | Key | Default | Extra |
+-----------+----------+------+-----+---------+-------+
| kodStud   | int(11)  | NO   | PRI | 0       |       |
| fam       | char(20) | YES  |     | NULL    |       |
| imya      | char(15) | YES  |     | NULL    |       |
| otch      | char(20) | YES  |     | NULL    |       |
| gr_nomber | int(11)  | YES  |     | NULL    |       |
+-----------+----------+------+-----+---------+-------+
5 rows in set (0.00 sec)

mysql> _
```

Рис. 1.3

просмотр структуры таблицы

осуществляется командой **DESCRIBE** или **Desc:**

DESCRIBE *table [column]*

DESC *table [column]*

Имя столбца (не обязательное) может содержать знаки подстановки SQL, в этом случае будет выдана информация о всех подходящих столбцах (рис. 1.4).

```
MySQL Command Line Client

mysql> desc student imya;
+-------+----------+------+-----+---------+-------+
| Field | Type     | Null | Key | Default | Extra |
+-------+----------+------+-----+---------+-------+
| imya  | char(15) | YES  |     | NULL    |       |
+-------+----------+------+-----+---------+-------+
1 row in set (0.01 sec)

mysql> describe student imya;
+-------+----------+------+-----+---------+-------+
| Field | Type     | Null | Key | Default | Extra |
+-------+----------+------+-----+---------+-------+
| imya  | char(15) | YES  |     | NULL    |       |
+-------+----------+------+-----+---------+-------+
1 row in set (0.00 sec)

mysql> desc student;
+-----------+----------+------+-----+---------+-------+
| Field     | Type     | Null | Key | Default | Extra |
+-----------+----------+------+-----+---------+-------+
| kodStud   | int(11)  | YES  | UNI | NULL    |       |
| fam       | char(20) | YES  |     | NULL    |       |
| imya      | char(15) | YES  |     | NULL    |       |
| otch      | char(20) | YES  |     | NULL    |       |
| gr_nomber | int(11)  | YES  |     | NULL    |       |
+-----------+----------+------+-----+---------+-------+
5 rows in set (0.02 sec)

mysql> _
```

Рис. 1.4

Создать таблицу Prep с ключевым полем KodPrep.

Установить следующие типы остальных полей:

- famPrep – char(20);
- ImyaPrep – char(15);
- OtchPrep, kaf, dolzh, stepen, zvanie – char(20).

Создать таблицу Dis с ключевым полем KodDis.

Установить следующие типы остальных полей:

- dis_name – char(20);
- hours – integer;
- DopOtch_sem – integer;
- VidDopOtch – char(20);
- Semester – integer;
- VidOtch – char(20);
- KodPrep – integer.

Создать таблицу ZanStud с ключевым полем KodZanStud.

Установить следующие типы остальных полей:

- KodStud, KodDis, KodPrep, Ball_KurRab, Ball – integer;
- Test – char(15).